

1 Смесь экспертов (MoE)

1.1 Роль FFN

Механизм внимания позволяет выявить зависимости между токенами. Это помогает модели понять контекст, но для корректной генерации следующего токена одного контекста недостаточно. Модель должна извлекать информацию, заложенную в её собственные веса.

Именно этим и занимается FFN-блок. В FFN входной токен сначала проецируется в пространство большей размерности:

$$z_1 = xW_1$$

Каждый столбец матрицы весов W_1 можно рассматривать как представление некоторой выученной моделью семантической концепции. Скалярное произведение токена x на такой столбец будет показывать, насколько хорошо x соответствует этой концепции.

Очевидно, что токен x может соответствовать концепциям, выученным слоем W_1 , а может не соответствовать. Чтобы отсеять нерелевантные результаты, мы используем функцию активации.

С этой точки зрения для фильтрации результата прекрасно подойдет рассмотренная ранее функция SwiGLU, поскольку она сама обучается, какие результаты релевантны, а какие - нет.

$$z_2 = \text{SwiGLU}(x)$$

Вектор z_2 имеет большую размерность, чем x , поэтому мы проецируем его в пространство исходной размерности

$$y = z_2W_2$$

Можно представить, что столбцы этой матрицы кодируют результаты соответствия токена x концепциям первого слоя в представление, удобное для дальнейшей обработки моделью.

Таким образом, если механизм внимания работает с контекстом каждого токена, то FFN блок хранит знания модели о мире и определяет семантику токена.

1.2 Разреженность

Получается, что если мы хотим улучшить общее знание модели о мире, то необходимо увеличить количество хранимых на каждом слое семантических концепций, то есть увеличить размерность W_1 , и, как следствие, W_2 .

Но здесь мы упираемся в инженерный потолок: увеличение размеров FFN приведет к сильному росту количества вычислений. Также, эти параметры и активации раздутого вектора z_2 необходимо где-то хранить, а это увеличивает потребление памяти.

Однако, конкретный токен вряд ли будет соответствовать всем выученным моделью семантическим концепциям на текущем слое. Это значит, что большинство результатов будут нерелевантны, а значит будут отсеяны функцией активации.

В таком случае возникает справедливый вопрос: а зачем нам их считать и хранить? Хотелось бы исключить ненужные операции и не считать то, что всё равно потом будет занулено.

Чтобы решить эту проблему — разобьем небольшую сеть FFN на множество мелких сетей — **экспертов**. Каждый токен будем обрабатывать лишь небольшим количеством наиболее релевантных экспертов.

Такой подход называют **разреженностью**: каждый отдельный токен обрабатывается лишь частью параметров слоя, хотя для обработки всех токенов в достаточно большой последовательности по итогу будут задействованы все параметры слоя.

1.3 Маршрутизатор

Чтобы определить, какой токен каким экспертом будет обработан, мы используем отдельную сеть — **маршрутизатор**.

По сути, маршрутизатор решает задачу классификации: для каждого токена вычисляется вектор логитов, который обрабатывается функцией Softmax для получения распределения вероятностей.

Из этого распределения мы выбираем k самых больших значения и отправляем токены на соответствующие этим значениям k экспертов.

Объединение малых экспертных сетей с сетью-маршрутизатором называется смесью экспертов (Mixture of Experts, MoE).

Обучение этих сетей, само собой, выполняется с помощью обратного распространения. С экспертами всё понятно: они просто обрабатывают свою подпоследовательность токенов. А как обучать роутер? Ведь операция отбора k экспертов (TopK(p)) — недифференцируема.

Чтобы обучить маршрутизатор с помощью обратного распространения, будем умножать выход выбранных экспертов на соответствующую этому эксперту вероятность роутера.

Итого, MoE-слой, который возможно обучить, будет выглядеть следующим образом:

$$y = \sum_{i \in \text{TopK}(p)} E_i(x) \cdot p_i(x),$$

где $p(x)$ — распределение наиболее релевантных экспертов для токена x , построенное роутером.

Маршрутизатор мы можем построить, используя один полносвязный слой. Столбцы матрицы этого слоя можно представить как векторы, описывающие семантическую особенность каждого эксперта. Чем больше токен будет сонаправлен с таким вектором, тем больше будет результат их ска-

лярного умножения и тем выше значение вероятности после применения Softmax.

1.4 Мелкозернистые эксперты

Чем больше экспертов, тем больше знаний модель хранит о мире и тем больше у нее параметров.

Как мы ранее обсуждали в контексте законов масштабирования, чем больше параметров в модели, тем ниже лосс после обучения на фиксированном объеме данных (при условии, что не возникает переобучения или недообучения).

МоЕ позволяет масштабировать модель на порядки: с сотен миллиардов параметров до триллионов, сохраняя при этом такое же количество требуемых FLOPs для обучения.

Для этого значительно увеличим количество экспертов, но при этом активными, то есть теми экспертами, которые обрабатывают конкретный токен, оставим лишь малую часть. Например, всего оставим активными 8 из 256 экспертов, как было сделано в DeepSeek v3.

Такие слои МоЕ называются мелкозернистыми (fine-grained MoE).

1.5 Общие эксперты

Мелкозернистые эксперты во время обучения получают экспертность в определенных областях. Однако, токены часто содержат в себе некоторую универсальную информацию, которую так или иначе придётся обрабатывать всем экспертам, что по итогу "размывает" их экспертность.

Выделим среди них один или несколько экспертов, которые гарантированно будут обрабатывать каждый токен:

$$\text{MoE}(x) = \sum_{i \in \text{TopK}(p)} E_i(x) \cdot p_i(x) + \sum_{j=1}^{N_{\text{shared}}} E_{\text{shared}j}(x),$$

таким эксперты называются **общими** (shared).

Общие эксперты берут на себя задачу по обработке универсальной информации, позволяя остальным экспертам сильнее специализироваться в своей области.

2 Экспертный параллелизм

Мы разобрали идею МоЕ, теперь нужно понять, как решить инженерную задачу по ее реализации. Основная задача МоЕ-слоя — масштабирование количества параметров.

Конечно, мы можем использовать тензорный параллелизм для масштабирования, однако, в таком случае на каждом GPU будет находится по доле

всех экспертов, а это означает усложнение логики и увеличение количества пересылаемых данных.

Как мы ранее обсуждали, гораздо проще не делить одного эксперта на N_{GPU} частей, а разделить всех экспертов на N_{GPU} групп и каждую такую группу расположить на отдельном девайсе. Это не освобождает нас от тяжелых коммуникаций, но по крайней мере упрощает логику.

Такой способ параллелизма называется **экспертным параллелизмом (Expert Parallelism, EP)**. Так, пусть в MoE-слое есть 64 экспертов и степень параллелизма экспертов равна 8 ($EP=8$). В простейшем случае эксперты равномерно раскладываются по GPU, то есть на каждом ранге EP будет по 8 экспертов.

2.1 Отправка токенов к экспертам (Dispatch)

После работы маршрутизатора, каждый токен получает набор экспертов $\{E_j\}_{j=1}^k$, на которые должен отправиться

$$x_t \rightarrow E_{i_1}, E_{i_2}, \dots, E_{i_k}$$

Если эти эксперты расположены на других GPU (а скорее всего эти именно так), то токен нужно отправить на другие девайсы. Операция пересылки токенов на GPU, где хранятся назначенные эксперты, называется **dispatch**.

Сначала токены необходимо сгруппировать по целевым девайсам. Этот шаг называется локальной перестановкой (local permute).

Операция коммуникации, которая лежит в основе dispatch — *All-to-All*. Каждый девайс отправляет сгруппированные токены по целевым девайсам (не отправляя только те, для которых этот девайс — целевой), параллельно с этим принимая токены с других девайсов.

2.2 Локальное вычисление экспертов (Grouped GEMM)

После dispatch каждый девайс получает набор токенов для своих локальных экспертов. Каждый эксперт — маленький FFN-блок. В наивной реализации мы бы запускали отдельный FFN для каждого эксперта. Но это неэффективно.

Во-первых, разные эксперты получают разное число токенов. Один эксперт может получить большое количество данных, другой — малое, третий — вообще ни одного (хотя такого сценария лучше избегать).

Во-вторых, каждый эксперт — это два небольших GEMM-кernels с эпилогами (dropout, функция активации и прочие операции, которые легко объединить с тяжелым матричным умножением). GPU плохо утилизирует ресурсы при выполнении малых матричных умножений, не стоит также забывать про накладные расходы на запуск kernelов.

Поэтому логичным шагом будет сгруппировать вычисления экспертов. Один из простых подходов — использовать **Batched GEMM** — в этом случае вычисления нескольких экспертов объединяются в один batched-вызов.

Однако, в этом случае придется падить меньшие матрицы токенов до самой большой матрицы.

Чтобы избежать этого можно использовать более специализированный подход — **Grouped GEMM**. Это один кернел, внутри которого исполняется много независимых GEMM-задач.

Внутри кернела есть планировщик, который в зависимости от вычисляемого тайла (вернее, в зависимости от идентификатора блока) выходной матрицы определяет тайлы входных матриц, которые необходимо перемножить друг с другом.

2.3 Возврат и объединение результатов (Combine)

После локального вычисления экспертов нуно вернуть результаты обратно. Этот этап называется сборкой (**combine**). **Combine** — обратная операция по отношению к **dispatch**.

Следующий слой трансформера, после MoE, ожидает, что токены будут расположены в исходном порядке и на нужных девайсах. Поэтому результаты нужно переслать обратно и восстановить нужную структуру батча.

Для этого, с помощью второго *All-to-All* выходы экспертов возвращаются на те девайсы, где должны находиться соответствующие токены. На данный момент токены сгруппированы по экспертам, поэтому чтобы вернуть их в предыдущий порядок необходимо использовать операцию, обратную **permute**. Чтобы оптимизировать вычисления на GPU и не запускать множество мелких кернелов, объединим эту операцию с умножением на веса маршрутизатора и суммированием между экспертами.

Стоит отметить, что *All-to-All* — тяжелая коммуникация, которую к тому же не спрятать под вычисления. Накладные расходы экспертного параллелизма, использующего 2 такие операции, — необходимая цена за масштабирование.

2.4 Capacity

В идеальном случае маршрутизатор равномерно распределяет токены между экспертами. Тогда каждый эксперт получает примерно одинаковое число токенов. На практике, роутер склонен к тому, чтобы распределение токенов между экспертами было вырожденным

В самом начале обучения веса инициализируются случайным образом. Ни о каких семантических концепциях в экспертах или маршрутизаторе пока и речи нет. Эксперты выбираются случайным образом. Из-за этого, один эксперт может получить больше токенов и, как следует, лучше обучиться. Благодаря этому, на следующем шаге роутер с большей вероятностью выберет именно этот эксперт, из-за чего он еще лучше обучится и на новом шаге роутер с еще большей вероятностью снова выберет его.

Это замкнутый круг, вернее даже — система с положительной обратной связью. По итогу распределение токенов наиболее вероятно станет вырож-

денным, что приведет к тому, что один девайс будет вынужден принять больше токенов, чем способен вместить.

Чтобы избежать ошибки *CUDA out of memory*, в МоЕ вводится понятие **ёмкости эксперта (capacity)** — максимальное число токенов, которое эксперт может обработать в данном батче.

$$\text{Capacity}(E) = \left\lceil \frac{T \cdot k}{N_{\text{experts}}} \cdot C \right\rceil,$$

где T — количество токенов в батче, k — количество активных экспертов и C — коэффициент ёмкости. Если $C = 1$, то каждый эксперт получает ровно столько токенов, сколько назначается на один токен в среднем, остальные токены не обрабатываются. Если $C > 1$, то эксперту дается небольшой запас.

Ёмкость экспертов также важна и для экспертного параллелизма, потому что позволяет заранее определить размеры буферов коммуникации, аллокации памяти и так далее. Если коэффициент C — большой, то буферы раздуваются, если маленький — повышается риск переполнения.

3 Балансировка нагрузки

Необработка токенов экспертами ухудшает обучение, поэтому чтобы избежать этого необходимо балансировать нагрузку между экспертами.

3.1 Ауксилиарный лосс

Важность каждого эксперта можно определить, просуммировав соответствующие вероятности по каждому токеноу в батче:

$$\text{Importance}(E_i) = \sum_{x \in B} p_i(x)$$

Вычислим коэффициент вариации по полученному вектору важности:

$$V(\text{Importance}) = \frac{\sigma(\text{Importance})}{\frac{1}{N} \sum_{i=1}^N \text{Importance}_i}$$

Чем ниже V , тем равномернее распределение важности. Если $V = 0$, то распределение полностью равномерное. Однако полностью равномерное распределение важности не гарантирует полностью равномерное распределение нагрузки на экспертов.

Так что будем также напрямую измерять нагрузку на каждого эксперта:

$$\text{Load}(E_i) = \sum_{x \in B} 1\{i \in \text{TopK}(p(x))\}$$

Но эта величина — дискретная, а это значит, что мы не можем использовать ее для вычисления обратного распространения ошибки.

Добавим к логитам роутера шум:

$$\tilde{s}_i(x) = s_i + \varepsilon_i \sigma_i(x), \quad \varepsilon_i \sim \mathcal{N}(0, 1)$$

и

$$\sigma_i(x) = \text{Softplus}(xW_{\text{noise}}), \quad \text{Softplus}(x) = \ln(1 + \exp(x))$$

$\sigma_i(x)$ — коэффициент масштабирования шума.

В таком случае мы можем считать нагрузку как

$$\text{Load}(E_i) = \sum_{x \in B} \text{Pr}(x, i),$$

где $\text{Pr}(x, i)$ — вероятность, что эксперт i будет выбран с учетом шума.

Выведем таким образом **вспомогательную функцию потерь (auxiliary loss)** для балансировки нагрузки экспертов:

$$L_{LB} = w_{\text{Importance}} V^2(\text{Importance}) + w_{\text{Load}} V^2(\text{Load})$$

Выглядит сложно, попробуем упростить. Сначала вычислим долю токенов, назначенных на эксперта i :

$$f_i = \frac{1}{T} \sum_{x \in B} 1_{i \in \text{TopK}(p(x))}$$

Затем вычислим доли вероятностей выбора маршрутизатора:

$$P_i = \frac{1}{T} \sum_{x \in B} p_i(x)$$

Эти два значения можно объединить в единую функцию потерь:

$$L_{LB} = \alpha \cdot N \sum_{i=1}^N f_i \cdot P_i$$

f_i — величина недифференцируемая, зато P_i — дифференцируемая. Поэтому мы можем избежать использования шума в логитах роутера.

3.2 Балансировка без ауксилиарного лосса

Однако добавление вспомогательной функции потерь снижает стабильность обучения, поскольку при неверном выборе коэффициента α функция потерь балансировки нагрузки может начать преобладать над функцией потерь основной задачи.

Попробуем балансировать нагрузку без вспомогательных лоссов.

Для начала, применим к логитам роутера не Softmax, а сигмоиду

$$p(x) = \sigma(xW_r)$$

$p(x)$ в данном случае перестает быть распределением вероятностей, но оставим запись в старой форме. Пусть маршрутизатор смещен в сторону выбора первых двух экспертов. Они получают больше токенов, чем остальные.

Для устранения дисбаланса добавим смещение к $p(x)$, зависящее от отклонения от среднего значения количества направленных на него токенов. При этом выходы экспертов всё еще умножаются на несмещённые оценки роутера.